

SAHO – Product Requirements Document

Phase 3 – Software Architecture & Engineering Specification

3.1 Purpose

This document defines the production-grade software architecture for SAHO. It establishes engineering standards, project structure, service boundaries, and implementation rules that Gemini Antigravity should follow while generating code.

3.2 Technology Stack

Layer	Technology
Framework	Next.js App Router + React 19 + TypeScript
Styling	Tailwind CSS + shadcn/ui + Framer Motion
State	Zustand + React Query
Backend	Firebase Authentication, Firestore, Cloud Functions
AI	Google Gemini Flash (Text + Vision)
Testing	Vitest, React Testing Library, Playwright
Deployment	Vercel + Firebase Hosting

3.3 Feature-Based Folder Structure

src/ app/ components/ features/ saho-now/ caregiver/ trigger-vision/
substance-identifier/ education/ recovery/ timeline/ circle-of-safety/ services/ ai/
firebase/ hooks/ store/ types/ utils/ constants/ tests/

3.4 Engineering Principles

- Strict TypeScript enabled.
- Feature-first architecture.
- SOLID principles.
- Reusable UI components only.
- No business logic inside UI components.
- AI requests isolated behind a service layer.
- Every async action must support loading, success and error states.

Component Hierarchy

App ── Layout ── Navigation ── SAHO NOW ── Caregiver ── Trigger Vision ──
Recovery Timeline ── Education Hub ── Settings

3.5 State Management

Global state: • Authentication • User profile • Emergency contacts • Theme • Accessibility preferences
Feature state: • Voice session • Camera session • Recovery session • AI responses

3.6 Service Layer

Every external interaction passes through dedicated services: • AIService • SpeechService • VisionService • EmergencyService • ContactService • EducationService UI components must never communicate directly with Firebase or Gemini.

3.7 Firestore Collections

users/ sessions/ caregiver_contacts/ recovery_logs/ education_progress/ settings/ audit_logs/

3.8 Authentication

Firebase Authentication using Email, Google Sign-In and anonymous guest mode for emergency access. Guest users may use crisis features without creating an account. Persistent recovery history requires authentication.

3.9 AI Architecture

Speech → SpeechService → AIService → Gemini Camera → VisionService → Gemini
Vision Gemini returns structured JSON validated using Zod before rendering.

3.10 Error Handling

• Global error boundary. • Retry transient failures. • Offline fallback. • User-friendly error messages. • Structured logging.

3.11 Security Requirements

• Environment variables for all secrets. • Firebase Security Rules. • Content Security Policy. • Input validation. • Prompt injection mitigation. • Image size/type validation. • Rate limiting for AI endpoints.

3.12 Performance Targets

• Lighthouse >95 • First Contentful Paint <1.8s • AI response target <3s • Lazy loading • Dynamic imports • Optimized images • PWA offline caching

3.13 Testing Strategy

Unit: utilities, hooks, services. Integration: AI workflows, authentication. E2E: complete recovery journey, caregiver flow, camera flow. Accessibility: WCAG 2.2 AA verification.

3.14 Repository Standards

Single public branch. Conventional commits. README with setup instructions. ESLint + Prettier. Husky pre-commit hooks. CI pipeline executing lint, tests and build.

3.15 Acceptance Criteria

Architecture is approved when:

- All features follow the feature-first structure.
- No duplicated business logic exists.
- Services are reusable.
- Strong typing is maintained.
- Security rules are implemented.
- Tests execute successfully.
- Performance and accessibility goals are achieved.